TruthLens

The project explained, start to finish

What it is, what it actually does, and how every part works, in plain English.

The whole project in one sentence

TruthLens seals a photo with cryptography **the moment it is taken** and writes that seal to a **blockchain**, so that later anyone can upload any image and find out whether it is the untouched original, an edited copy (and **where** it was edited), or something with no proven origin at all.

Sarthak Sanjay Kandhare · T.Y. Computer Science and Engineering · NMIET, Pune

1. The problem it solves

Today anyone can make a fake photo. AI tools can generate faces and scenes that look completely real ("deepfakes"), and simple editing apps can remove a person, change a background or paste in an object in seconds. So when you see a photo of an accident, a protest or a damaged car, you can no longer be sure it shows what really happened.

Most solutions try to **detect fakes**: an AI model looks at the pixels and guesses "this is 87% likely to be fake". That approach has three weaknesses:

- It is only ever a **guess** (a probability), never a proof.
- Every new AI generator needs the detector to be retrained, so it is always one step behind.
- Compressing or resizing an image (WhatsApp does this) destroys the clues the detector relies on.

2. The idea behind TruthLens

TruthLens flips the question. Instead of asking *"is this image fake?"*, it asks *"can this image prove where it came from?"*

> **An everyday comparison**
> Think of a sealed envelope with a wax stamp. You can't tell by looking at a letter whether it is genuine, but if the seal is unbroken and matches the sender's stamp, you know it came from them and nobody opened it. TruthLens puts a digital seal on a photo at the moment of capture. A real photo keeps its seal. An edited photo has a broken seal. A deepfake never had a seal at all.

This turns an impossible problem (spotting every fake) into a solvable one (checking a seal), using tools that are mathematically proven: hash functions, digital signatures and a blockchain.

3. What it actually does

The app does three jobs:

Job	What happens	Page in the app
1. Seal	You take a photo in the TruthLens camera. It is fingerprinted, signed by your device and recorded on the blockchain within about a second.	Capture
2. Verify	Anyone uploads any image. TruthLens checks it against the blockchain and gives one of three verdicts.	Verify
3. Track history	If a sealed photo is edited and the edit is sealed too, the chain original → edit → edit-of-edit is recorded and can be followed.	Ledger, Provenance timeline

The three verdicts

Verdict	Meaning
AUTHENTIC	Bit-for-bit identical to a sealed photo, and the device signature is valid. Not a single pixel has changed since it was taken.
EDITED	It looks like a sealed photo, but the file is different. TruthLens finds the original, shows a heatmap of the changed regions and the percentage modified.
UNVERIFIED	No record exists. Its origin cannot be proven. **This does not mean it is fake**, only that it was never sealed by TruthLens.

4. The three building blocks

Everything in TruthLens rests on three ideas. Once you understand these, the rest is just plumbing.

4.1 SHA-256: the photo's fingerprint

A **hash function** turns any file into a short, fixed-length code (64 hex characters for SHA-256). Two properties make it useful:

- **Change one pixel and the code changes completely.** There is no “slightly different” hash.
- It is practically impossible to make a different file with the same code.

So if a photo's SHA-256 matches the one recorded at capture, the photo is exactly the same, bit for bit.

4.2 Digital signature: proof of which device took it

When TruthLens first runs on a phone or laptop, it creates a pair of keys using **ECDSA P-256**, the same kind of cryptography used in banking and passkeys:

- The **private key** stays inside the browser and is marked *non-extractable*: the app can ask it to sign, but no script, not even TruthLens itself, can read or copy it.
- The **public key** is shared. Anyone can use it to check a signature, but nobody can use it to create one.

The device signs the photo's fingerprint. A valid signature proves that this exact photo was sealed by that specific device.

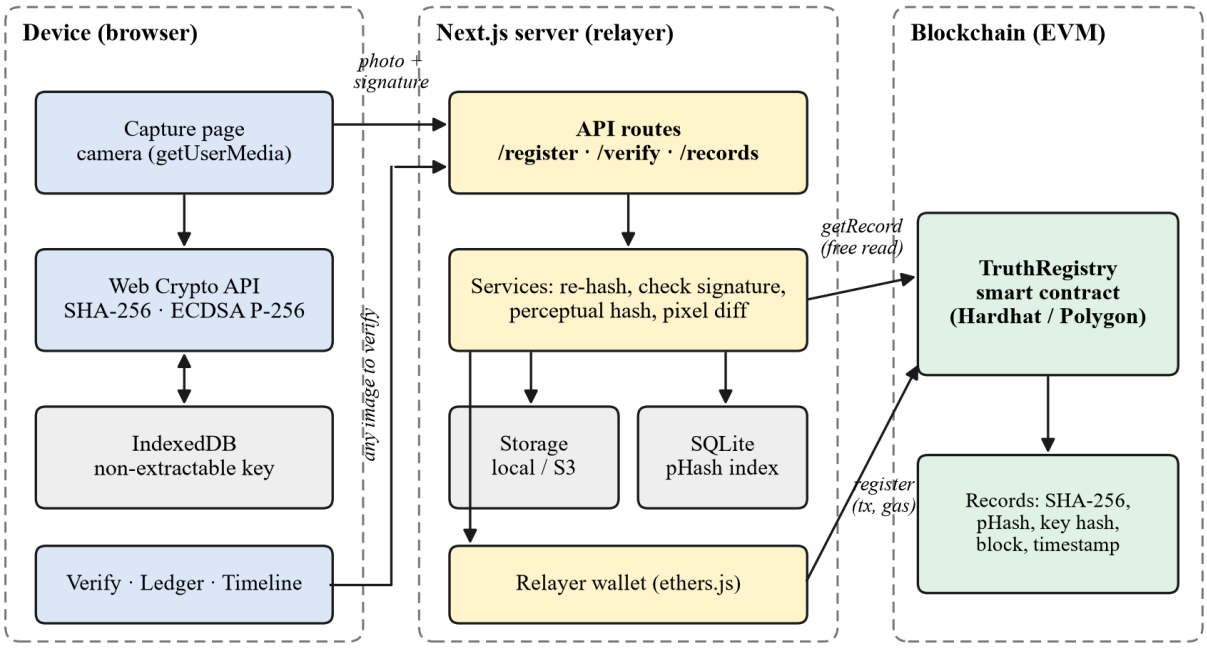
4.3 Blockchain: a public notebook nobody can erase

A blockchain is a shared record book. Once something is written, nobody (not even the person running TruthLens) can quietly change it or backdate it, and anyone can read it. TruthLens uses a small program on the blockchain called a **smart contract** (`TruthRegistry.sol`, written in Solidity) to store one record per sealed photo.

> **Only fingerprints go on the blockchain, never the photo**
> Storing a photo on a blockchain would be very expensive and would make private photos public. A 32-byte fingerprint proves the photo just as strongly. The photo itself is stored separately on the server (or in Amazon S3).

Stored in each on-chain record	Why
SHA-256 of the photo	The exact fingerprint (the record's key)
Perceptual hash (pHash)	A “looks-like” fingerprint, used to find the original of an edited copy
Fingerprint of the device's public key	Ties the record to the device that signed it
Storage link	Where the original photo is kept
Block number and timestamp	Set by the blockchain itself, so the time cannot be faked
Parent hash	For edits: which photo this one was edited from

5. The architecture: three parts

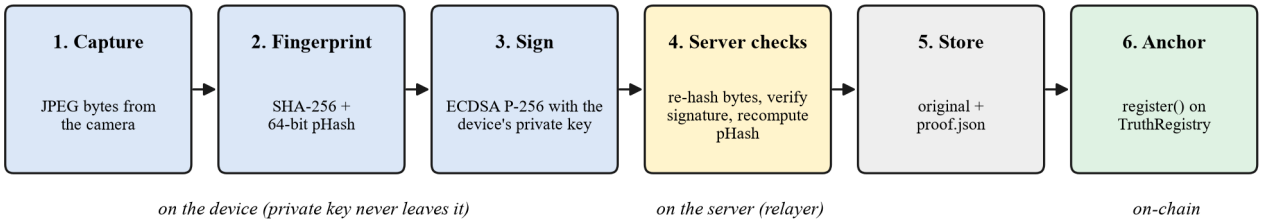


The three parts of TruthLens and how they talk to each other

Part	What it does
Device (any browser on a phone or laptop)	Opens the camera, calculates the fingerprints, signs them with the private key, and shows the Verify, Ledger and Timeline pages.
Server (Next.js app)	Receives the photo, **re-checks everything** the browser claims, stores the original, keeps a fast search index, and sends the blockchain transaction.
Blockchain (Hardhat locally, Polygon Amoy online)	Holds the TruthRegistry smart contract: the permanent, public source of truth.

> **Why is there a server in the middle? (the "relayer")**
> Writing to a blockchain costs a small fee called **gas**, paid from a crypto wallet. Ordinary users don't have wallets. So the server pays the gas with its own wallet and submits the record for them. The contract only accepts records from approved relayers, and the relayer only accepts photos with a valid device signature. Users need no wallet and pay nothing.

6. How sealing a photo works, step by step



From shutter press to on-chain record

Step	Where	What happens
1. Capture	Device	You press the shutter. The camera frame is saved as a JPEG file.
2. Fingerprint	Device	The browser computes the photo's SHA-256 and its 64-bit perceptual hash.
3. Sign	Device	The private key signs the message "`TruthLens/capture/v1`" + `SHA-256`. The prefix means this signature can never be reused to sign anything else.
4. Server checks	Server	The server **trusts nothing**: it re-calculates the SHA-256 from the bytes it received, checks the signature, recalculates the perceptual hash itself, and refuses duplicates.
5. Store	Server	The original photo and a proof file (public key + signature) are saved.
6. Anchor	Blockchain	The server calls `register()` on the contract. The blockchain records it in a block with its own timestamp.

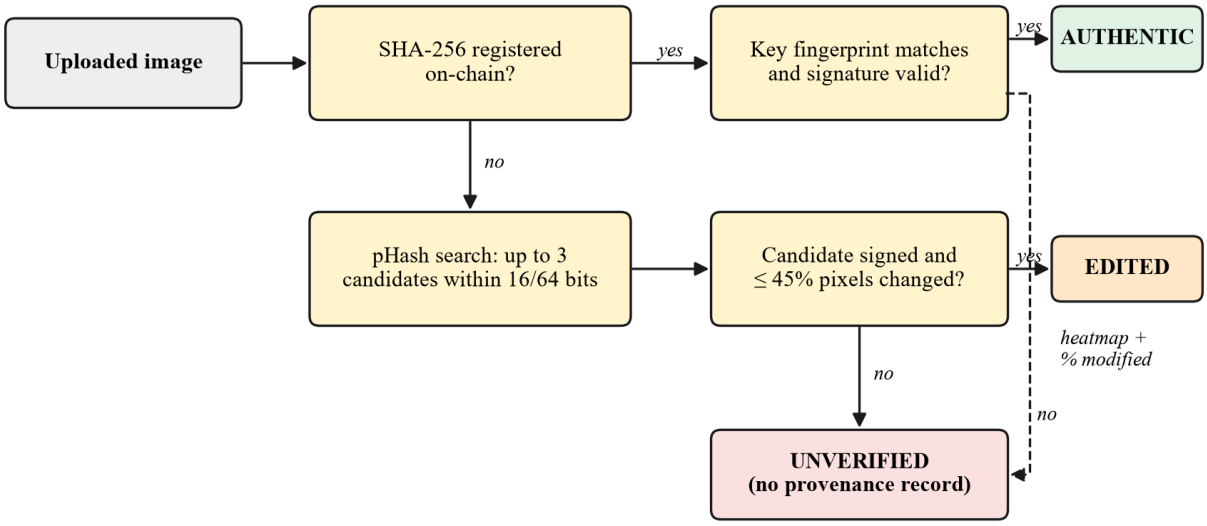
The app then prints a "receipt of seal" showing each step's real timing, the transaction hash and the block number.



The capture page after sealing: the receipt of seal

> **A detail worth knowing for the viva**
> The server saves the photo **before** writing to the blockchain. If the blockchain write fails, you just have an unused file. The other order could leave a permanent on-chain record pointing at a photo that doesn't exist.

7. How verifying an image works



The verification decision flow

When you drop an image on the Verify page, TruthLens asks two questions in order:

Question 1: is this exact file on the blockchain?

It calculates the file's SHA-256 and looks it up. If found, it checks that the stored public key matches the fingerprint on the blockchain and that the signature is valid. If both pass: **AUTHENTIC**.

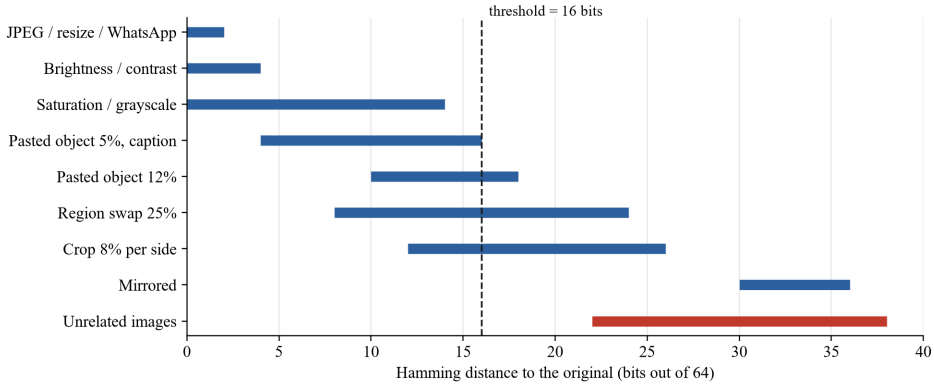
Question 2: does it look like a sealed photo?

If the exact file isn't found, it may still be an edited or re-compressed copy. A normal hash can't help here (one changed pixel changes it completely), so TruthLens uses a second kind of fingerprint:

> **Perceptual hash (pHash): a "looks-like" fingerprint**
> The image is shrunk to 32×32 grey pixels, converted with a mathematical transform (the DCT, the same one JPEG uses), and boiled down to 64 bits that describe its overall shape. Similar-looking images get similar bits. Two images count as "the same picture" if at most **16 of the 64 bits** differ.

TruthLens finds up to 3 sealed photos within that distance, then compares each one pixel by pixel. If at most 45% of the image changed, the verdict is **EDITED**, and the closest original is shown. Otherwise: **UNVERIFIED**.

Why 16 bits? It was measured, not guessed



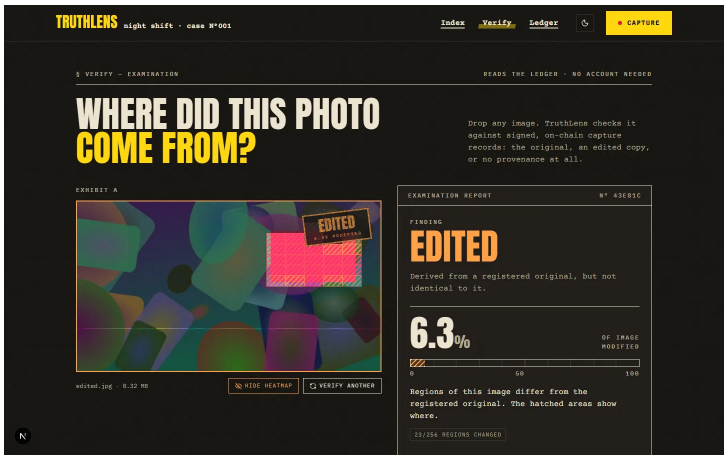
Measured distances: harmless changes stay below 16, unrelated images are all above 22

Re-compressing, resizing or WhatsApp-style sharing changes at most 2 bits. Unrelated photos differ by at least 22 bits. The threshold of 16 sits safely in between.

How the tamper heatmap is made

Once the original is found, both images are compared to show **where** the edit happened:

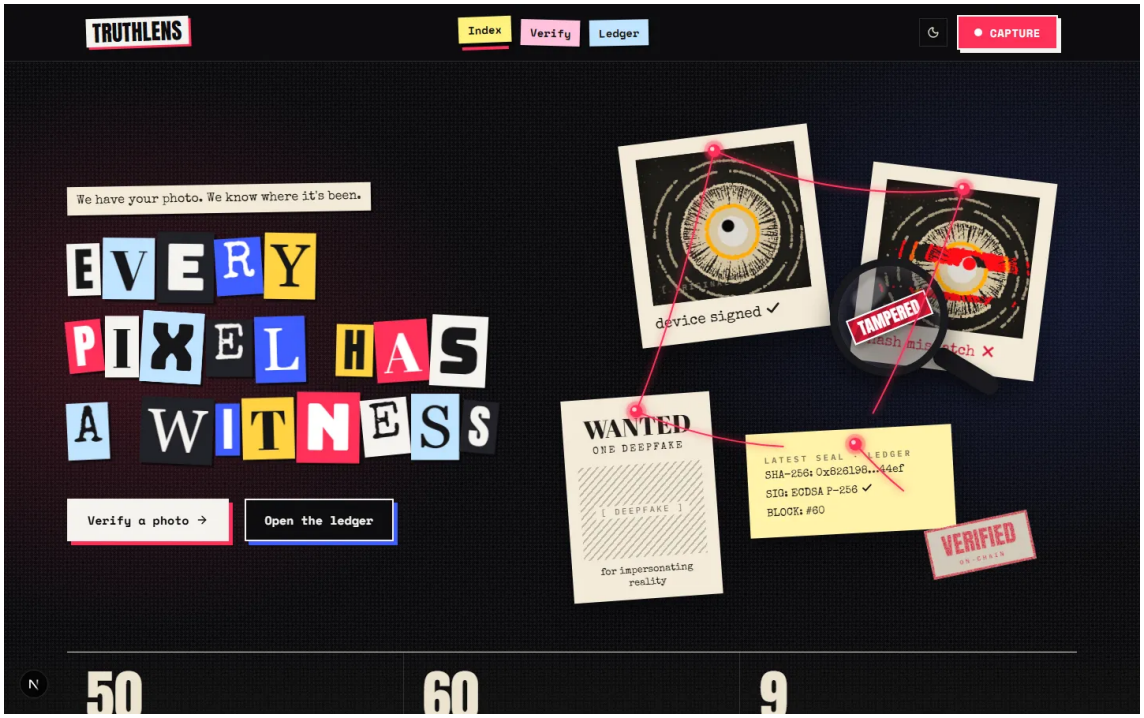
- Both are resized to 512 pixels wide, turned grey and slightly blurred (this hides harmless JPEG noise).
- Brightness and contrast are matched, so a whole-photo filter doesn't count as an edit.
- Any pixel that differs by more than 20 brightness levels counts as changed.
- The result is shown on a 16×16 grid: changed tiles are cross-hatched, and the report gives the % modified.



An EDITED verdict: the pasted region is hatched and 6.3% of the image is reported as modified

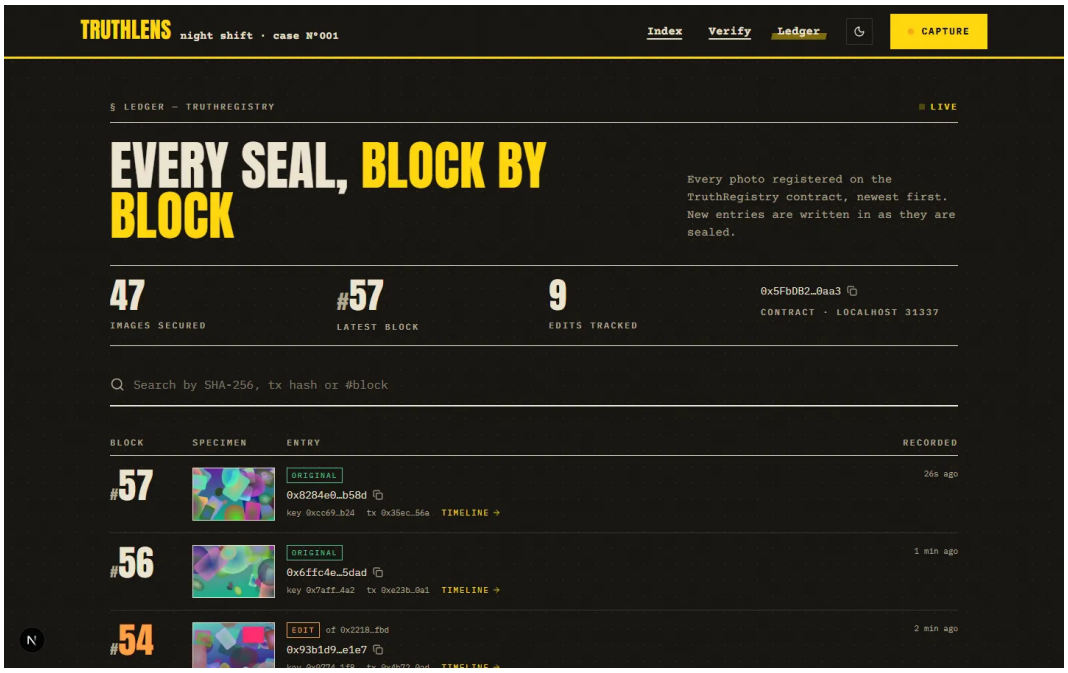
What was done to the photo	Reported as modified
Re-compressed, resized or sent through WhatsApp	0.0–0.3%
Brightness, contrast or a colour filter	0.0–0.2% (flagged as a global adjustment)
Caption text added	2.3–2.7%
Object pasted over 12% of the area	10.9–12.3%
A completely different photo	52.8–59.1% (above 45%, so not called an edit)

8. A tour of the app



Index (home): the idea as an evidence board, with live counters read from the blockchain

Page	What you can do there
Index	Explains the idea. The counters (images secured, blocks written, edits tracked) and the latest seal card are read live from the blockchain.
Capture	Full-screen camera. Press the shutter to seal a photo, then download it or verify it.
Verify	Drop or paste any image to get AUTHENTIC, EDITED or UNVERIFIED, with a magnifier and a before/after slider for edits.
Ledger	A block explorer listing every sealed photo, newest first, searchable by hash or block number.
Provenance timeline	The life of one photo: original, then every registered edit, as a film strip.



The Ledger: every seal, block by block

9. What TruthLens can and cannot prove

Being honest about the limits is part of the design, and examiners like it.

It CAN prove	It CANNOT prove
This exact file was signed by one specific device, no later than a certain block.	That the scene itself was real (someone could photograph a screen showing a deepfake).
Whether a copy was re-compressed, filtered or edited, and which regions changed.	Whether an image with no record is AI-generated. It is simply UNVERIFIED; there is no AI detector.
The whole chain of registered edits back to the original.	Anything about photos that were never captured with TruthLens.

Other known limits:

- Malware already on the phone could use the key (it still can't copy it). Hardware attestation is future work.
- Heavy crops and mirrored images may not be matched to their original.
- The server (relayer) decides what gets written, although it can never change past records.

10. How we know it works

Test suite	Count	What it checks
Smart contract tests	22	Registering, duplicates rejected, edit chains, only approved relayers can write
Unit tests	62	SHA-256 against official NIST values, signatures, perceptual hash, heatmap accuracy
End-to-end test	21 checks	The whole system, including attacks: forged signatures, tampered uploads
Browser walkthrough	4 modes	The full demo in a real browser on desktop, phone, dark mode and reduced motion

Cost: sealing one photo uses about 175,000 gas, roughly 0.005 POL on Polygon at 30 gwei. **Verifying is free**, because it only reads the blockchain.

11. Technologies used

Area	Technology	Used for
Smart contract	Solidity, Hardhat 3	The TruthRegistry contract, local blockchain, tests
Blockchain access	ethers.js	The relayer wallet sending transactions
Web app	Next.js 16, React 19, TypeScript	Pages and API routes in one project
Cryptography	Web Crypto API	SHA-256 and the non-extractable ECDSA P-256 key
Image processing	sharp	Decoding, perceptual hash, heatmap, thumbnails
Storage	Local folder or Amazon S3, SQLite	Original photos and a fast search index
Interface	Tailwind CSS, Framer Motion, three.js	Design, animations, the 3D eye
Testing	Mocha, Vitest, Playwright	Contract, unit, end-to-end and browser tests

12. Quick answers for the viva

Q. Why not just use an AI deepfake detector?

Detectors give a probability and break on new generators. A matching hash plus a valid signature is a mathematical proof. The two can work together: detectors for unsealed images, TruthLens for sealed ones.

Q. Why put it on a blockchain instead of a normal database?

Because nobody, including me as the operator, can secretly edit or backdate a record, and anyone can check it independently.

Q. Why P-256 and not Ethereum's own curve?

Browsers and phone security chips support P-256 natively, which lets the private key be non-extractable.

Q. What stops someone faking the fingerprint?

The server re-calculates every hash from the actual file and checks the signature against it.

Q. Does UNVERIFIED mean fake?

No. It only means there is no record. TruthLens proves provenance; it does not judge unsealed images.

13. How to run it

```
npm run chain
npm run deploy:local
npm run dev
npm run seed
```

Run the first line in one terminal and leave it open, the others in a second terminal, from the project folder. Then open **http://localhost:3000**.